

Cryptographic Hash Functions

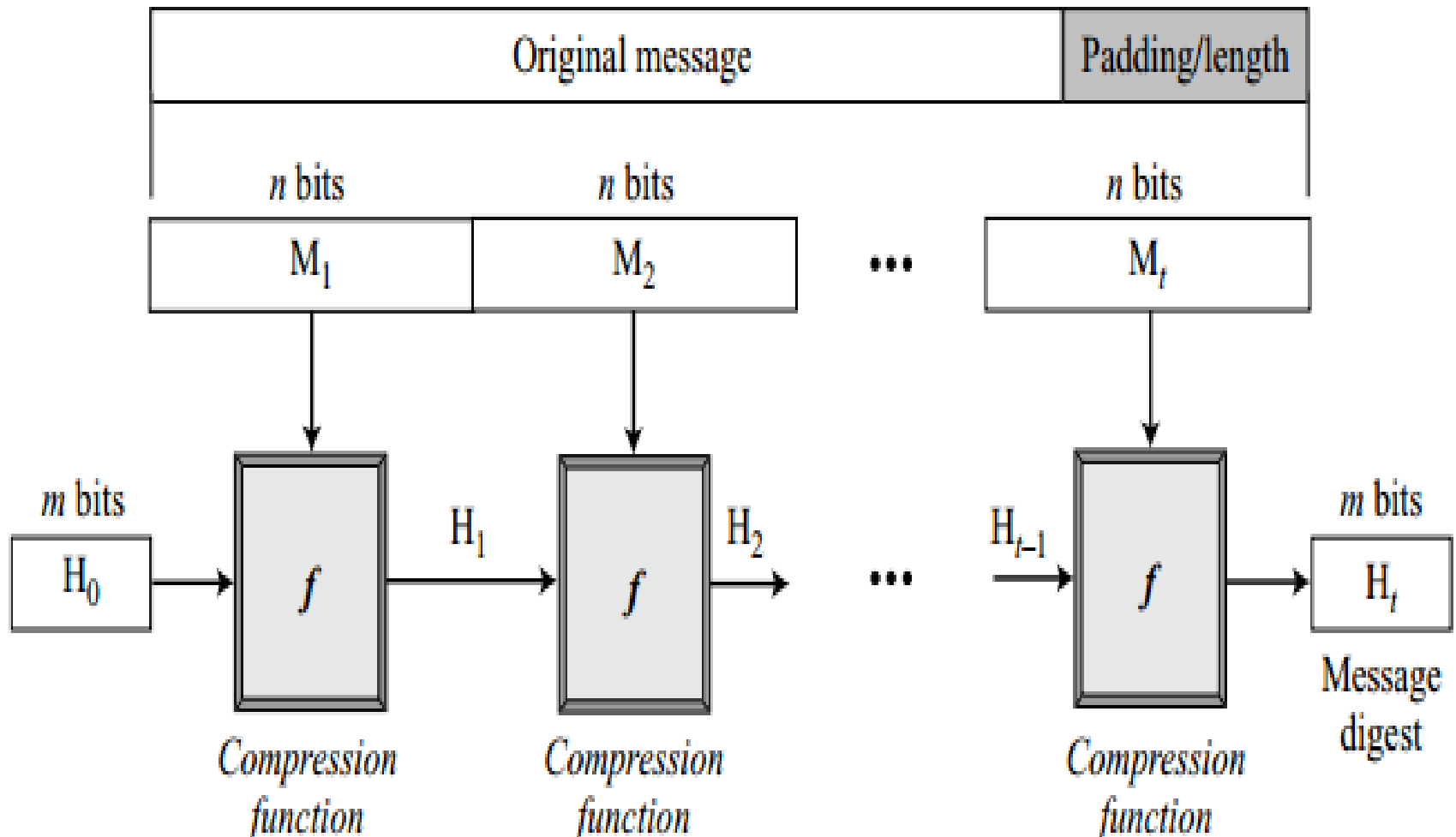
Introduction

- A **cryptographic hash function** takes a message of arbitrary length and creates a message digest of fixed length.
- Creating such a function is best accomplished using **iteration**.
 - a function (**compression function**) with fixed-size input is created and is used a necessary number of times.

Iterated Hash Function

- A **compression function** compresses an n -bit string to create an m -bit string where n is normally greater than m .
 - Also referred to as an **iterated cryptographic hash function**.
 - The Merkle-Damgard scheme is the basis for many cryptographic hash functions used today.
 - If the **compression function in the Merkle-Damgard scheme** is collision resistant, the hash function is also collision resistant.

Merkle-Damgard Scheme



Merkle-Damgard Scheme

The scheme uses the following steps:

1. The message length and padding are appended to the message to create an augmented message that can be evenly divided into blocks of n bits, where n is the size of the block to be processed by the compression function.
2. The message is then considered as t blocks, each of n bits. We call each block $M_1, M_2, \dots, M_t$. We call the digest created at t iterations $H_1, H_2, \dots, H_t$.
3. Before starting the iteration, the digest H_0 is set to a fixed value, normally called IV (initial value or initial vector).
4. The compression function at each iteration operates on H_{i-1} and M_i to create a new H_i . In other words, we have $H_i = f(H_{i-1}, M_i)$, where f is the compression function.
5. H_t is the cryptographic hash function of the original message, that is, $h(M)$.

Two Groups of Compression Functions

- Two different approaches are used in designing a hash function:
 - The compression function is made from scratch
 - it is particularly designed for this purpose.
 - Examples: MD family and SHA
 - A symmetric-key block cipher serves as a compression function.
 - Example: Rabin Scheme

Hash Functions Made from Scratch

- **Message Digest (MD)**
 - designed by Ron Rivest.
 - Several hash algorithms are MD2, MD4, and MD5
 - Among them, MD5 is strongest.
- **Secure Hash Algorithm (SHA)**
 - is a standard by NIST and published as FIP 180
 - Also referred to as Secure Hash Standard (SHS)
 - It is mostly based on MD5.
 - The standard was revised and produced as SHA-1, SHA-224, SHA-256, SHA-384, and SHA-512.

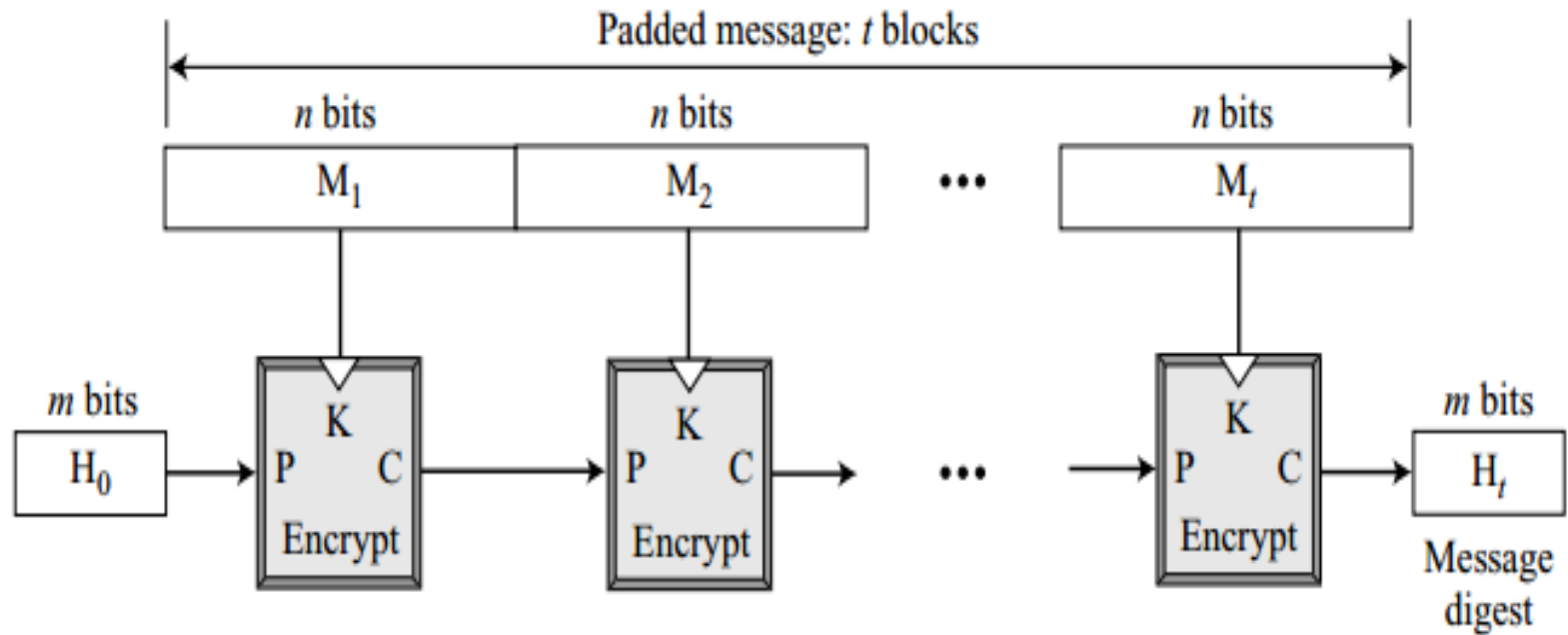
Characteristics of Secure Hash Algorithms (SHAs)

<i>Characteristics</i>	<i>SHA-1</i>	<i>SHA-224</i>	<i>SHA-256</i>	<i>SHA-384</i>	<i>SHA-512</i>
Maximum Message size	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size	512	512	512	1024	1024
Message digest size	160	224	256	384	512
Number of rounds	80	64	64	80	80
Word size	32	32	32	64	64

Hash Functions Based on Block Ciphers

- An iterated cryptographic hash function can use a symmetric-key block cipher as a compression function.
- **Rabin Scheme** is an iterated hash function based on the **Merkle-Damgard scheme**.
- The compression function is replaced by any encrypting cipher.
- The message block is used as the key; the previously created digest is used as the plaintext.

Rabin scheme

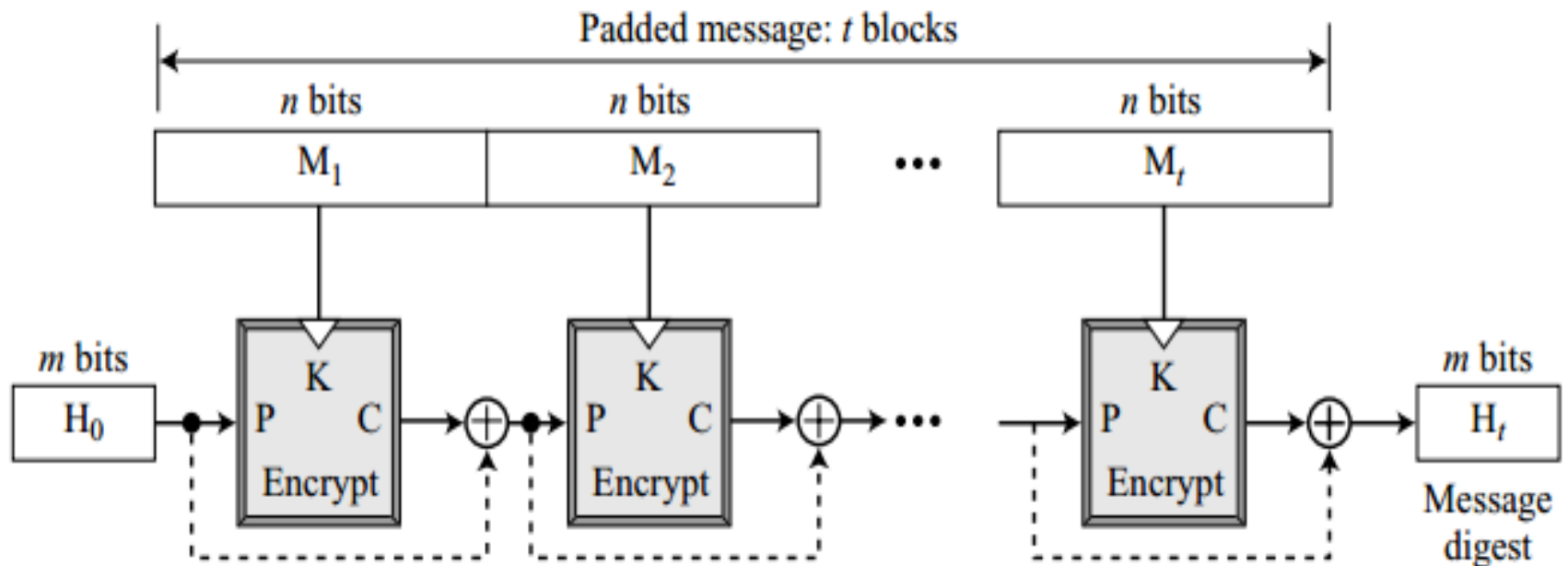


Rabin scheme

- The size of the digest is the size of data block cipher in the underlying cryptosystem.
 - For example, if DES is used as the block cipher, the size of the digest is only 64 bits.
 - Simple design but prone to meet-in-the-middle attack

Davies-Meyer Scheme

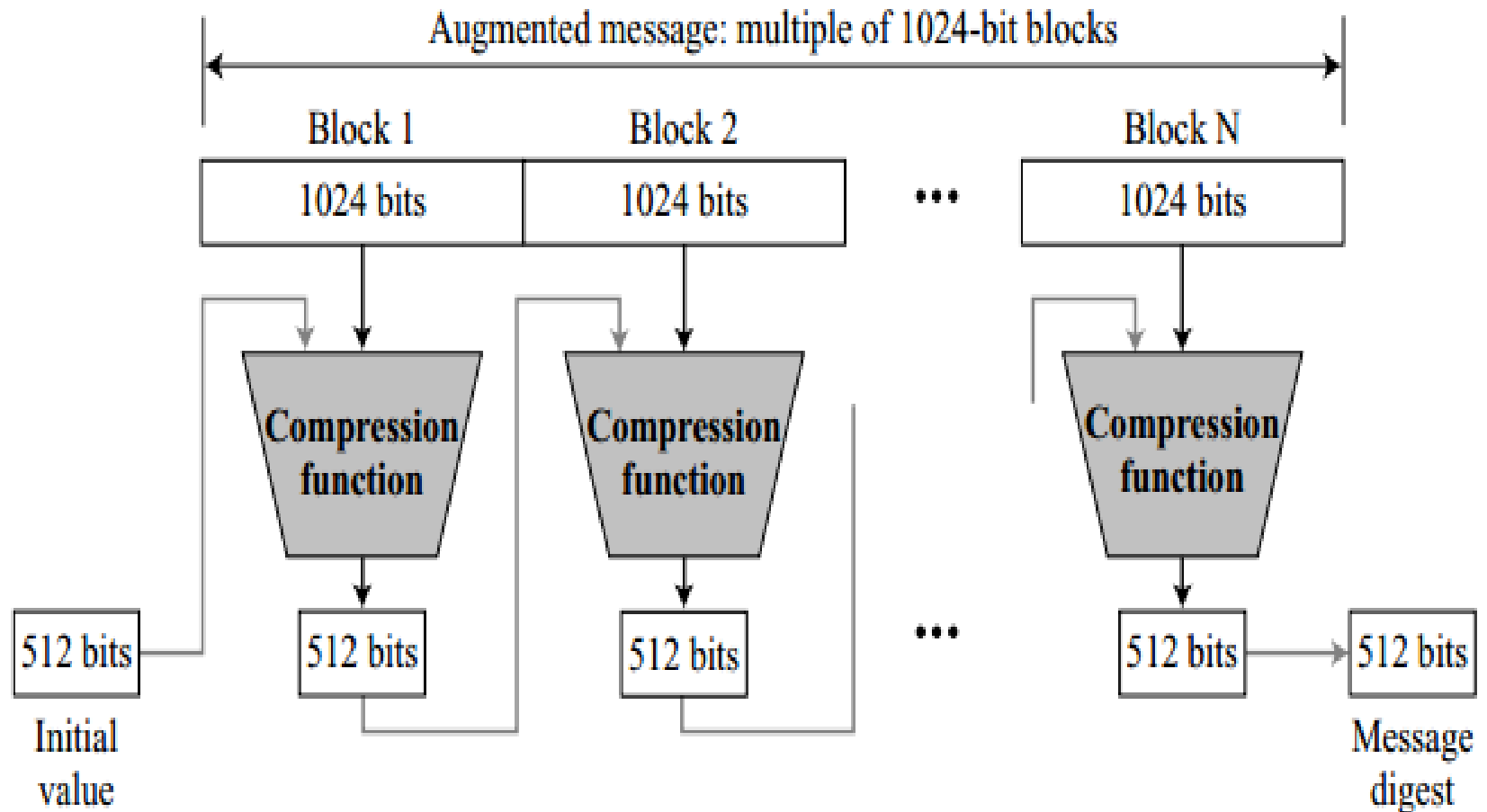
- It is same as the Rabin scheme except that it uses forward feed to protect against meet-in-the-middle attack.



SHA-512

- Based on the Merkle-Damgard scheme
- SHA-512 creates a digest of 512 bits from a multiple-block message.
- Each block is 1024 bits in length

Message Digest creation in SHA-512



Message Preparation

- SHA-512 insists that the length of the original message be less than 2^{128} bits.

Length Field and Padding

- Before the message digest can be created, SHA-512 requires the addition of a 128-bit unsigned-integer length field to the message
- This is the **length of the original message before padding**
- Before the addition of the length field, we need to pad the original message to make the length a multiple of 1024.

Length Field and Padding

- The length of the padding field can be calculated as follows:
Let $|M|$ be the length of the original message
 $|P|$ be the length of the padding field.

The format of the padding is one 1 followed by the necessary number of 0s.

$$(|M| + |P| + 128) = 0 \bmod 1024 \quad \rightarrow \quad |P| = (-|M| - 128) \bmod 1024$$

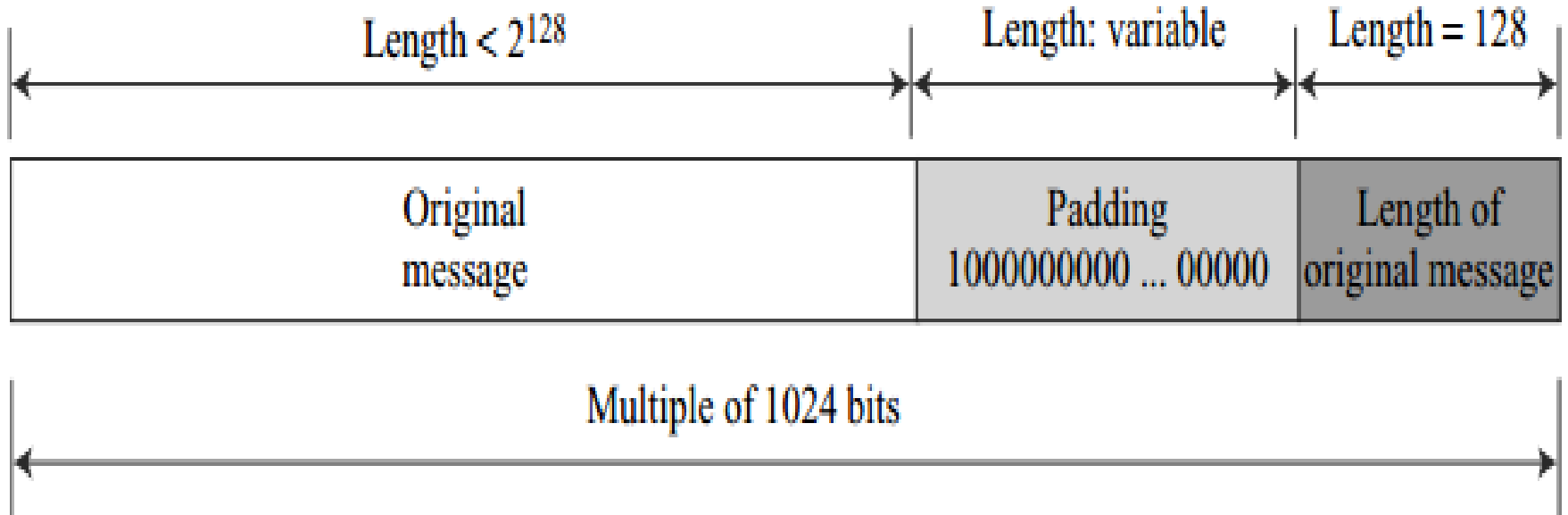
- What is the minimum number of padding bits that can be added to a message?

The minimum length of padding is 0 and it happens when $(-M - 128) \bmod 1024$ is 0.

This means that $|M| = -128 \bmod 1024 = 896 \bmod 1024$ bits.

The last block in the original message is 896 bits. We add a 128-bit length field to make the block complete.

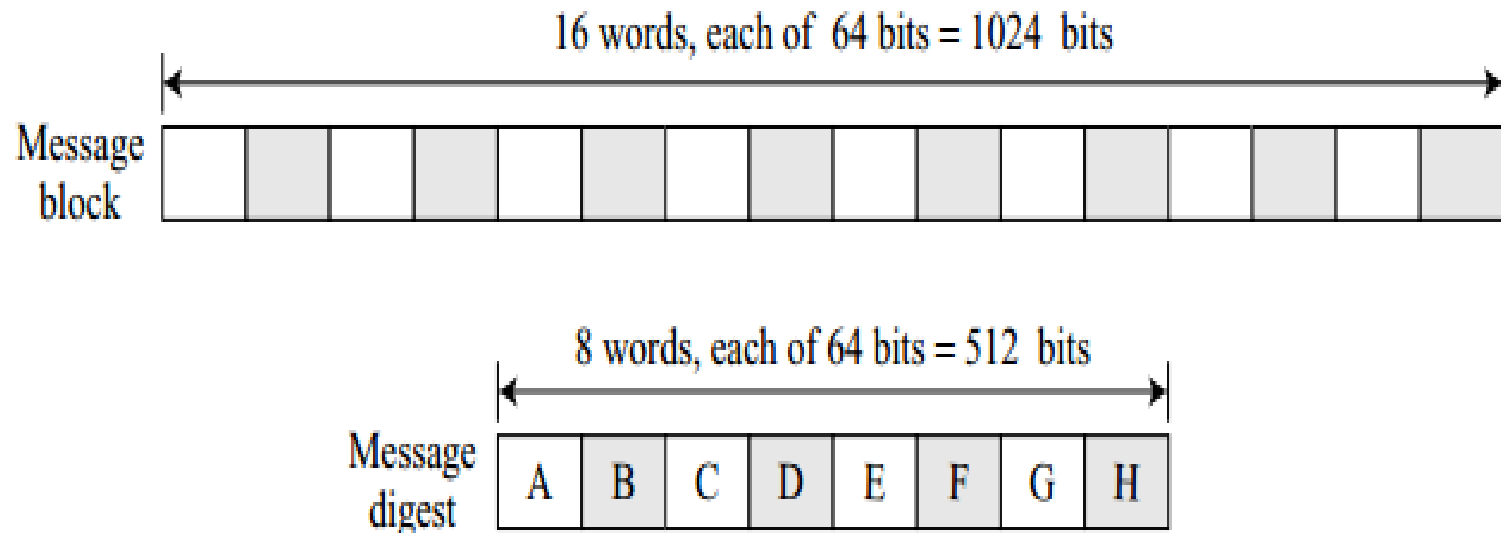
Padding and length field in SHA-512



Words

- SHA-512 operates on words; it is word oriented.
- A word is defined as 64 bits.
- After the padding and the length field are added to the message, each block of the message consists of sixteen 64-bit words.
- The message digest is only eight words and the words are named A, B, C, D, E, F, G, and H

A message block and the digest as words

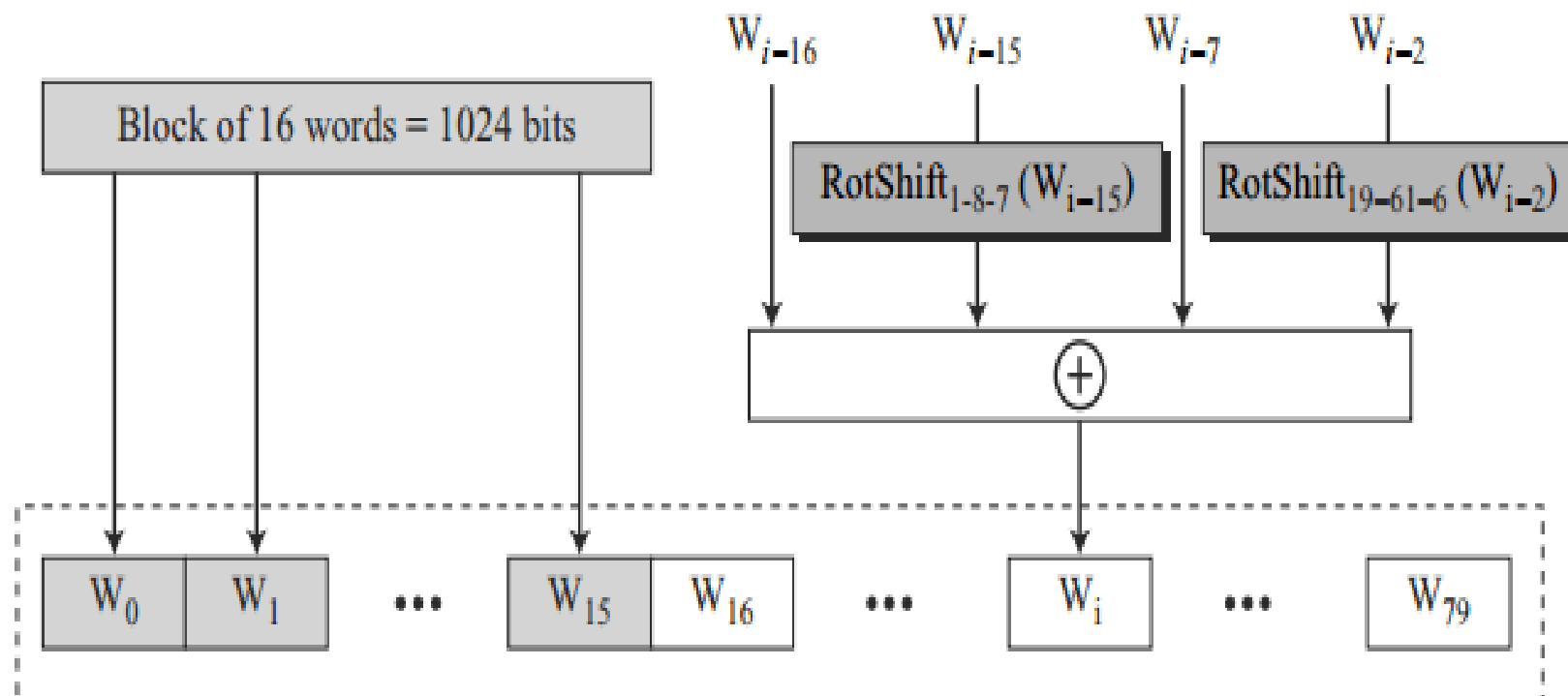


SHA-252 is word-oriented. Each block is 16 words; the digest is only 8 words.

Word Expansion

- Before processing, each message block must be expanded.
- The 16-word block needs to be expanded to 80 words, from W_0 to W_{79}
- The 1024-bit block becomes the first 16 words; the rest of the words come from already-made words

Word expansion in SHA-512



$$\text{RotShift}_{l-m-n}(x): \text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x)$$

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

$\text{ShL}_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

Example

- How W_{60} is made?
- Each word in the range W_{16} to W_{79} is made from four previously-made words.
- W_{60} is made as:

$$\text{RotShift}_{1-m-n}(x): \text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x)$$

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

$\text{ShL}_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

$$W_{60} = W_{44} \oplus \text{RotShift}_{1-8-7}(W_{45}) \oplus W_{53} \oplus \text{RotShift}_{19-61-6}(W_{58})$$

Message Digest Initialization

- The algorithm uses eight constants for message digest initialization (A_0 to H_0).
- Values of constants in message digest initialization of SHA-512 are:

<i>Buffer</i>	<i>Value (in hexadecimal)</i>	<i>Buffer</i>	<i>Value (in hexadecimal)</i>
A_0	6A09E667F3BCC908	E_0	510E527FADE682D1
B_0	BB67AE8584CAA73B	F_0	9B05688C2B3E6C1F
C_0	3C6EF372EF94F82B	G_0	1F83D9ABFB41BD6B
D_0	A54FE53A5F1D36F1	H_0	5BE0CD19137E2179

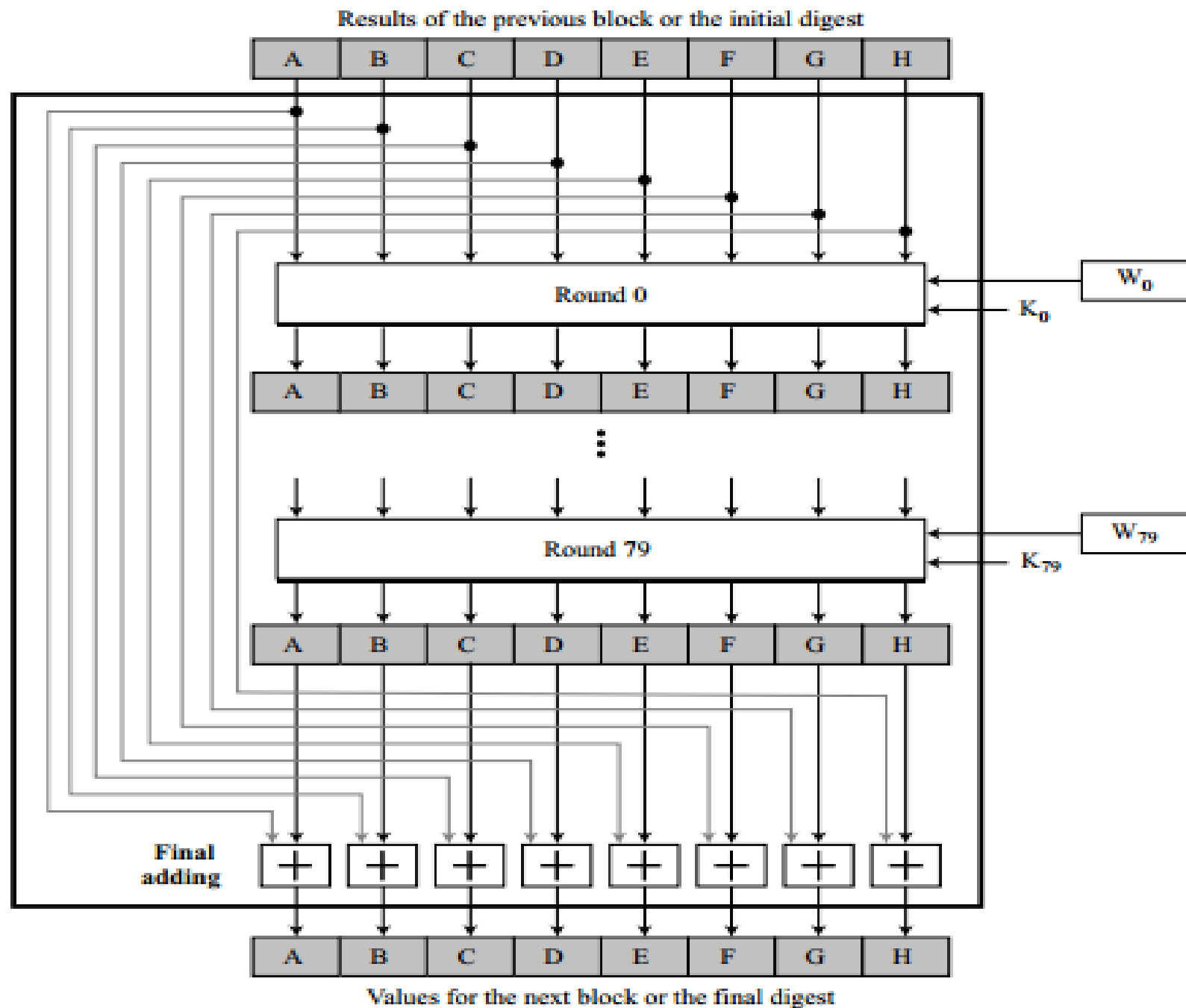
- The values are calculated from the first eight prime numbers (2, 3, 5, 7, 11, 13, 17, and 19).
- Each value is the fraction part of the square root of the corresponding prime number after converting to binary and keeping only the first 64 bits.
- For example, the eighth prime is 19, with the square root $\sqrt{19} = 4.35889894354$.
 - Converting the number to binary with only 64 bits in the fraction part, we get

$$(100.0101\ 1011\ 1110\ \dots\ 1001)_2 \rightarrow (4.5BE0CD19137E2179)_{16}$$

SHA-512 keeps the fraction part, $(5BE0CD19137E2179)_{16}$, as an unsigned integer.

Compression Function

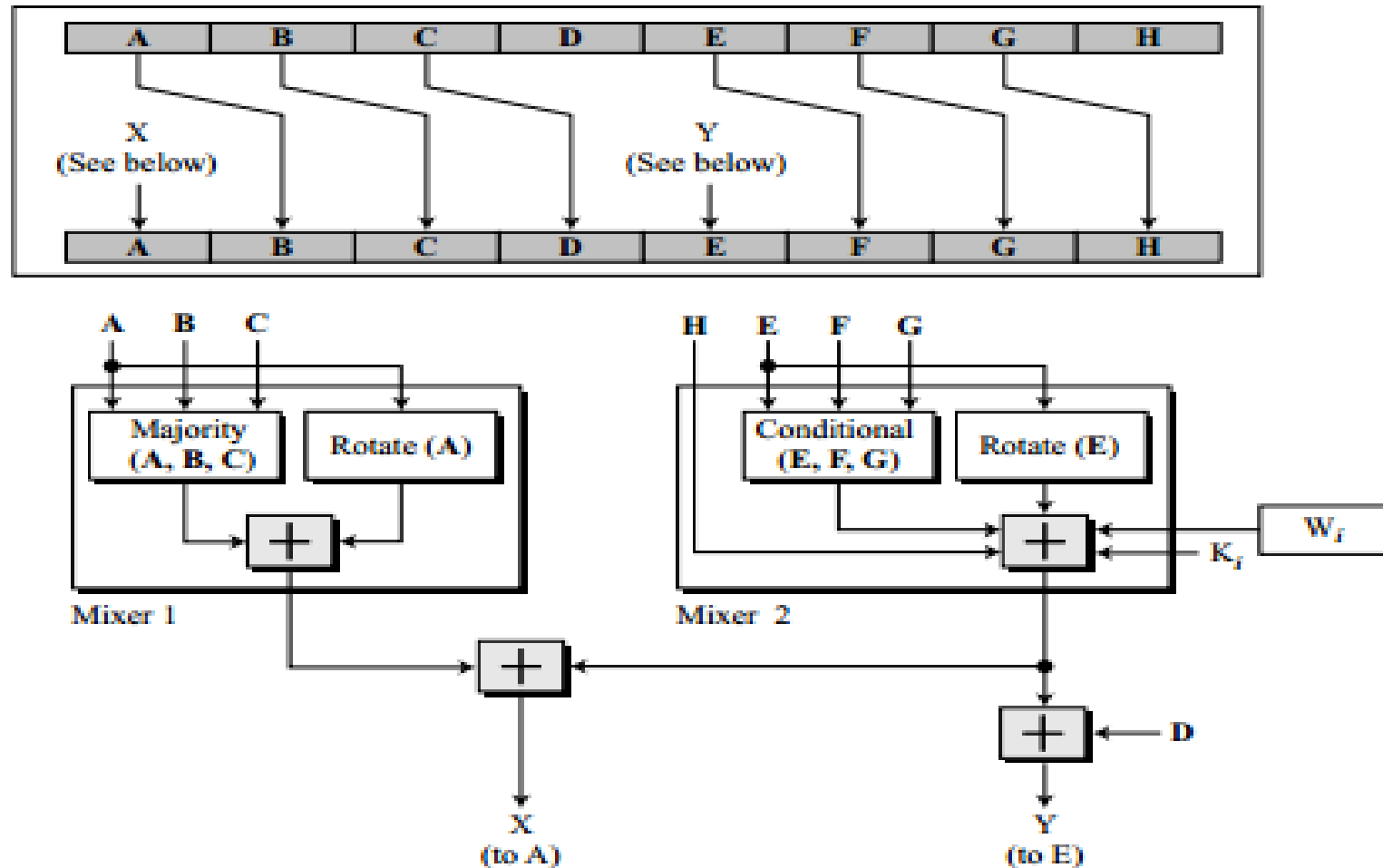
- The processing of each block of data in SHA-512 involves 80 rounds.
- In each round, the contents of eight previous buffers, one word from the expanded block (W_i), and one 64-bit constant (K_i) are mixed together and then operated on to create a new set of eight buffers.



Structure of Each Round

- In each round, eight new values for the 64-bit buffers are created from the values of the buffers in the previous round.
- There are two mixers, three functions, and several operators.

Round



Majority (x, y, z)

$$(x \text{ AND } y) \oplus (y \text{ AND } z) \oplus (z \text{ AND } x)$$

Conditional (x, y, z)

$$(x \text{ AND } y) \oplus (\text{NOT } x \text{ AND } z)$$

Rotate (x)

$$\text{RotR}_{28}(x) \oplus \text{RotR}_{34}(x) \oplus \text{RotR}_{39}(x)$$

$\oplus$ addition modulo 2^{64}

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

Structure of Each Round

- The **Majority function**, which is a bitwise function, takes three corresponding bits in three buffers (A, B, and C) and calculates $(A_j \text{ AND } B_j) \oplus (B_j \text{ AND } C_j) \oplus (C_j \text{ AND } A_j)$
- The **Conditional function**, also a bitwise function, takes three corresponding bits in three buffers (E, F, and G) and calculates $(E_j \text{ AND } F_j) \oplus (\text{NOT } E_j \text{ AND } G_j)$

Structure of Each Round

- The Rotate function
 - right-rotates the three instances of the same buffer (A or E) and applies the exclusive-or operation on the results.
Rotate (A): $\text{RotR28}(A) \oplus \text{RotR34}(A) \oplus \text{RotR29}(A)$
Rotate (E): $\text{RotR28}(E) \oplus \text{RotR34}(E) \oplus \text{RotR29}(E)$
- The right-rotation function, $\text{RotR}_i(x)$, right-rotates its argument i bits
- The addition operator used in the process is addition modulo 2^{64} .

Structure of Each Round

- There are 80 constants, K_0 to K_{79} , each of 64 bits
- These values are calculated from the first 80 prime numbers (2, 3,..., 409)
- Each value is the fraction part of the cubic root of the corresponding prime number after converting it to binary and keeping only the first 64 bits.
- Example:

80th prime is 409, with the cubic root $(409)^{1/3} = 7.42291412044$. Converting this number to binary with only 64 bits in the fraction part, we get

$$(111.0110\ 1100\ 0100\ 0100\ \dots\ 0111)_2 \rightarrow (7.6C44198C4A475817)_{16}$$

SHA-512 keeps the fraction part, $(6C44198C4A475817)_{16}$, as an unsigned integer.

Eighty constants used for 80 rounds in SHA-512

428A2F98D728AE22	7137449123EF65CD	B5C0FBCFEC4D3B2F	E9B5DBA58189DBBC
3956C25BF348B538	59F111F1B605D019	923F82A4AF194F9B	AB1C5ED5DA6D8118
D807AA98A3030242	12835B0145706FBE	243185BE4EE4B28C	550C7DC3D5FFB4E2
72BE5D74F27B896F	80DEB1FE3B1696B1	9BDC06A725C71235	C19BF174CF692694
E49B69C19EF14AD2	EFBE4786384F25E3	0FC19DC68B8CD5B5	240CA1CC77AC9C65
2DE92C6F592B0275	4A7484AA6EA6E483	5CB0A9DCBD41FBD4	76F988DA831153B5
983E5152EE66DFAB	A831C66D2DB43210	B00327C898FB213F	BF597FC7BEEF0EE4
C6E00BF33DA88FC2	D5A79147930AA725	06CA6351E003826F	142929670A0E6E70
27B70A8546D22FFC	2E1B21385C26C926	4D2C6DFC5AC42AED	53380D139D95B3DF
650A73548BAF63DE	766A0ABB3C77B2A8	81C2C92E47EDAEE6	92722C851482353B
A2BFE8A14CF10364	A81A664BBC423001	C24B8B70D0F89791	C76C51A30654BE30
D192E819D6EF5218	D69906245565A910	F40E35855771202A	106AA07032BBD1B8
19A4C116B8D2D0C8	1E376C085141AB53	2748774CDF8EEB99	34B0BCB5E19B48A8
391C0CB3C5C95A63	4ED8AA4AE3418ACB	5B9CCA4F7763E373	682E6FF3D6B2B8A3
748F82EE5DEFB2FC	78A5636F43172F60	84C87814A1F0AB72	8CC702081A6439EC
90BEFFFA23631E28	A4506CEBDE82BDE9	BEF9A3F7B2C67915	C67178F2E372532B
CA273ECEEA26619C	D186B8C721C0C207	EADA7DD6CDE0EB1E	F57D4F7FEE6ED178
06F067AA72176FBA	0A637DC5A2C898A6	113F9804BEF90DAE	1B710B35131C471B
28DB77F523047D84	32CAAB7B40C72493	3C9EBE0A15C9BEBE	431D67C49C100D4C
4CC5D4BECB3E42B6	4597F299CFC657E2	5FCB6FAB3AD6FAEC	6C44198C4A475817

Analysis

- With a message digest of 512 bits, SHA-512 expected to be resistant to all attacks, including collision attacks.